Multiple Linear Regression for Baseball Win Prediction

Homework #1

Naomi Buell Richie Rivera Alexander Simon Nana Frimpong
Said Naqwe

2026-03-01

1 Introduction

We analyze and model data on a professional baseball team from 1871 to 2006. Each record has the performance of the team for the given year, with all of the statistics adjusted to match the performance of a 162 game season. We build a multiple linear regression model on the training data to predict the number of wins for the team.

2 Methods

2.1 Data Exploration

2.1.1 Data Characteristics

The moneyball training data set contains 2276 rows and 17 features, and is 91.01 percent complete. The data are all numeric, and the target variable is `TARGET_WINS`, which represents the number of games won by the team. The other variables represent various performance statistics for the team. See Table 1 for descriptive characteristics of the training data. Variables range from 0% to 92% missing. We address missingness by dropping columns and imputing in the Data Preparation section.

```
tbl_summ_stats <- moneyball_train |> skim_without_charts() |> janitor::clean_names() |> as_tibl
    rename(Variable = skim_variable, "Completeness rate" = complete_rate, "Mean" = numeric_mea
    select(Variable, "Completeness rate", Mean, Min, P25, P50, P75, Max) |>
    mutate(across(where(is.numeric), ~ signif(., 2))) |> knitr::kable()

tbl_summ_stats
```

Table 1: Summary statistics for all variables in the training dataset

Variable	Completeness rate	Mean	Min	P25	P50	P75	Max
INDEX	1.000	1300	1	630	1300	1900	2500
TARGET_WINS	1.000	81	0	71	82	92	150
TEAM_BAT-TING_H	1.000	1500	890	1400	1500	1500	2600
TEAM_BAT-TING_2B	1.000	240	69	210	240	270	460
TEAM_BAT-TING_3B	1.000	55	0	34	47	72	220
TEAM_BAT-TING_HR	1.000	100	0	42	100	150	260
TEAM_BAT-TING_BB	1.000	500	0	450	510	580	880
TEAM_BAT-TING_SO	0.960	740	0	550	750	930	1400
TEAM_BASERUN_SB	0.940	120	0	66	100	160	700
TEAM_BASERUN_CS	0.660	53	0	38	49	62	200
TEAM_BAT-TING_HBP	0.084	59	29	50	58	67	95
TEAM_PITCH-ING_H	1.000	1800	1100	1400	1500	1700	30000
TEAM_PITCH-ING_HR	1.000	110	0	50	110	150	340
TEAM_PITCH-ING_BB	1.000	550	0	480	540	610	3600
TEAM_PITCH-ING_SO	0.960	820	0	620	810	970	19000
TEAM_FIELD-ING_E	1.000	250	65	130	160	250	1900
TEAM_FIELD-ING_DP	0.870	150	52	130	150	160	230

2.1.2 Correlation Matrix

```
# Get correlation matrix
moneyball_train_cor <- moneyball_train |>
    cor(use = "pairwise.complete.obs")
```

```
corrplot(moneyball_train_cor, method = "circle", type = "upper", tl.cex = 0.4, sig.level = 0.05
```

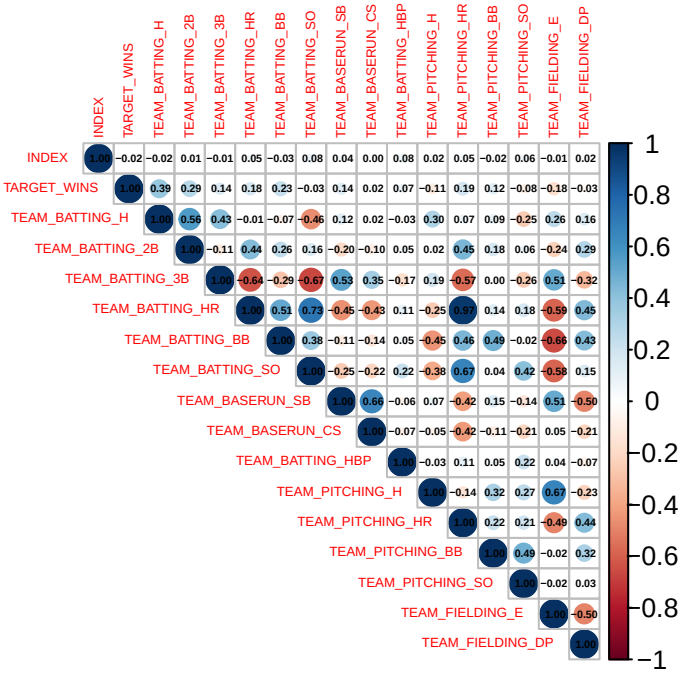


Figure 1: Correlation matrix of all variables in the training dataset

The correlation between all variables are shown Figure 1. The most correlated variables with the target variable `TARGET_WINS` are `TEAM_BATTING_H`, `TEAM_BATTING_2B`, and `TEAM_BATTING_BB`, which may be predictors of wins. The most correlated independent variables are `TEAM_BATTING_HR` and `TEAM_PITCHING_HR`, which makes sense as teams that hit many homeruns may also allow many homeruns, for example.

Figure 2 shows the distribution of z-scores of all variable observations, and identifies outliers in the training data. `TEAM-PITCHING_SO`, `TEAM_PITCHING_H`, and `TEAM_PITCHING_BB` have many high outliers. If there are bad leverage points in our models, we may need to create dummy variables for the outliers.

```
moneyball_train |>
    pivot_longer(cols = everything(), names_to = "Variable", values_to = "Value") |>
    group_by(Variable) |>
    mutate(Z_Score = scale(Value)[,1]) |>
    ungroup() |>
    ggplot(aes(x = Z_Score, y = Variable)) +
    geom_boxplot() +
    theme_minimal() +
    theme(panel.grid = element_blank(), axis.text.y = element_text(angle = 0)) +
```

```
    xlab("Z-Score")
```

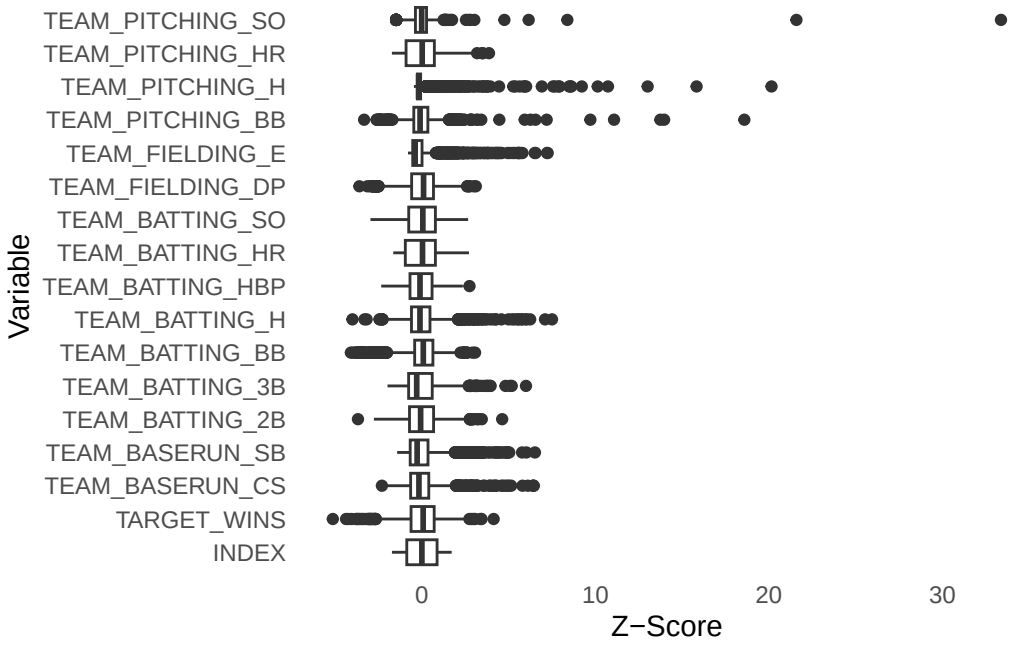


Figure 2: Box plots for identifying outliers in the training data

2.2 Data Preparation

We handled high missingness in the variables `TEAM_BATTING_HBP`, `TEAM_BASERUN_CS`, and `TEAM_FIELDING_DP` (91.6%, 33.9%, and 12.6% missing, respectively), which are not very correlated with the target variable, by excluding these variables. The remaining variables had less than 10% missing, so we will impute the missing values with the median of each variable. We avoid dropping `TEAM_BASERUN_SB` as it is the most correlated variable with the target variable that has missingness, and instead impute it.

The median imputation was picked based on the distributions of missing variables: `TEAM_BASERUN_SB` and `TEAM_PITCHING_SO` have a long right tail so the median is a better imputation method than the mean. Both variables have observed have extreme outliers pulling the mean towards a right direction compared to where most the data sits. When the distribution of variables reveals two distinct peaks as observed in `TEAM_BATTING_SO`, we can refer to this distribution as bimodal. Since the mean of bimodal falls between two peaks, this area is observed to be the underrepresented region of the missing data. The median imputation is utilized to approach one of the two clusters of missing variables with the largest distribution of missing variables. Below are the histograms for the distributions of these variables.

```
# Create density curve for TEAM_BASERUN_SB
p1 <- ggplot(moneyball_train, aes(x = TEAM_BASERUN_SB)) +
```

```
    geom_density() +
    theme_minimal() +
    theme(panel.grid = element_blank(), axis.text = element_text(size = 7), axis.title = eleme
    xlab("TEAM_BASERUN_SB") +
    ylab("Density")

# Create density curve for TEAM_PITCHING_SO
p2 <- ggplot(moneyball_train, aes(x = TEAM_PITCHING_SO)) +
    geom_density() +
    theme_minimal() +
    theme(panel.grid = element_blank(), axis.text = element_text(size = 7), axis.title = eleme
    xlab("TEAM_PITCHING_SO") +
    ylab("")

# Create density curve for TEAM_BATTING_SO
p3 <- ggplot(moneyball_train, aes(x = TEAM_BATTING_SO)) +
    geom_density() +
    theme_minimal() +
    theme(panel.grid = element_blank(), axis.text = element_text(size = 7), axis.title = eleme
    xlab("TEAM_BATTING_SO") +
    ylab("")

# Combine the histograms
patchwork::wrap_plots(p1, p2, p3, ncol = 3) + patchwork::plot_layout(guides = 'collect')
```

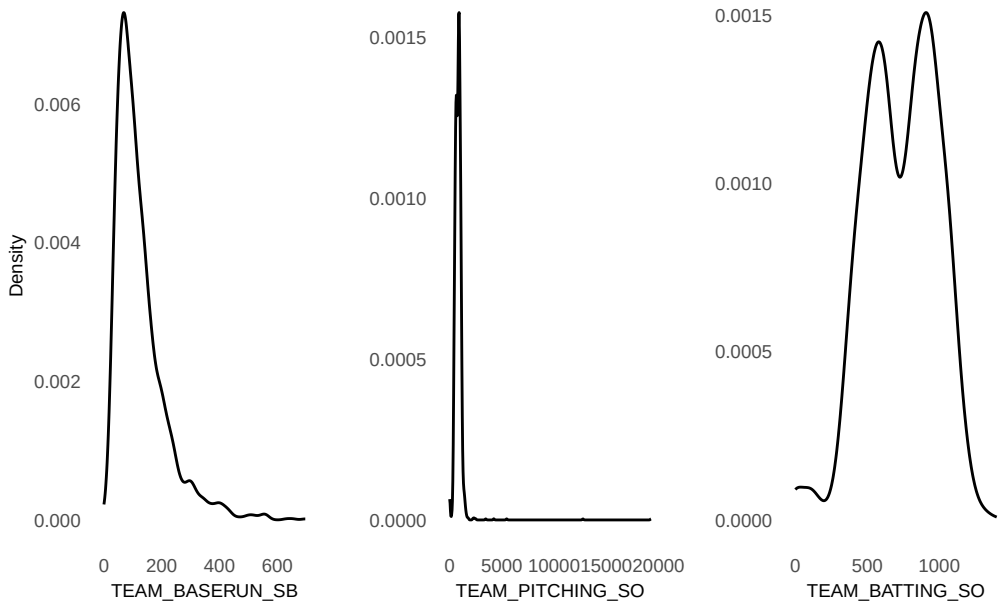


Figure 3: Histograms of variables with missingness selected for imputation

To begin by remove variables with high missingness variables and low correlation with `TARGET_WINS`: `TEAM_BATTING_HBP` (91.6% missing, r=0.07), `TEAM_BASERUN_CS` (33.9% missing, r=0.02), and `TEAM_FIELDING_DP` (12.6% missing, r=-0.03).

```
# Drop high missingness variables and low-correlation variables with the data
# These variables lack a strong relationship with response variable TARGET_WINS
moneyball_clean <- moneyball_train |>
  select(-TEAM_BATTING_HBP, -TEAM_BASERUN_CS, -TEAM_FIELDING_DP)

moneyball_eval_clean <- moneyball_eval |>
  select(-TEAM_BATTING_HBP, -TEAM_BASERUN_CS, -TEAM_FIELDING_DP)
```

Before our predictor variables `TEAM_BASERUN_CS`, `TEAM_BASERUN_SB` and `TEAM_PITCHING_SO` are imputed, a binary indicator is created to preserve information that the value was originally missing. Our indicator for each flag is (1 = missing, 0 = present).

```
moneyball_clean <- moneyball_clean |>
  mutate(
    TEAM_BASERUN_SB_MISSING  = as.integer(is.na(TEAM_BASERUN_SB)),
    TEAM_BATTING_SO_MISSING  = as.integer(is.na(TEAM_BATTING_SO)),
    TEAM_PITCHING_SO_MISSING = as.integer(is.na(TEAM_PITCHING_SO))
  )
```

```
moneyball_eval_clean <- moneyball_eval_clean |>
  mutate(
    TEAM_BASERUN_SB_MISSING  = as.integer(is.na(TEAM_BASERUN_SB)),
    TEAM_BATTING_SO_MISSING  = as.integer(is.na(TEAM_BATTING_SO)),
    TEAM_PITCHING_SO_MISSING = as.integer(is.na(TEAM_PITCHING_SO))
  )
```

Imputation is now based on the distribution of missing data of the following variables. `TEAM_BASERUN_SB`, `TEAM_PITCHING_SO`, and `TEAM_BATTING_SO`. `TEAM_BASERUN_SB` and `TEAM_PITCHING_SO` are right skewed, therefore they are imputed around the mean. `TEAM_BATTING_SO` is bimodal, so we can impute around the median to avoid placing the imputed values in region between two peaks where only a few observations exist.

```
moneyball_clean <- moneyball_clean %>%
  mutate(
    # Mean imputation for right-tailed variables
    TEAM_BASERUN_SB = ifelse(
      is.na(TEAM_BASERUN_SB),
      mean(TEAM_BASERUN_SB, na.rm = TRUE),
      TEAM_BASERUN_SB
    ),
    TEAM_PITCHING_SO = ifelse(
      is.na(TEAM_PITCHING_SO),
      mean(TEAM_PITCHING_SO, na.rm = TRUE),
      TEAM_PITCHING_SO
    ),

    # Median imputation for bimodal variable
    TEAM_BATTING_SO = ifelse(
      is.na(TEAM_BATTING_SO),
      median(TEAM_BATTING_SO, na.rm = TRUE),
      TEAM_BATTING_SO
    )
  )

moneyball_eval_clean <- moneyball_eval_clean %>%
  mutate(
    # Mean imputation for right-tailed variables
    TEAM_BASERUN_SB = ifelse(
      is.na(TEAM_BASERUN_SB),
```

```
      mean(TEAM_BASERUN_SB, na.rm = TRUE),
      TEAM_BASERUN_SB
    ),
    TEAM_PITCHING_SO = ifelse(
      is.na(TEAM_PITCHING_SO),
      mean(TEAM_PITCHING_SO, na.rm = TRUE),
      TEAM_PITCHING_SO
    ),

    # Median imputation for bimodal variable
    TEAM_BATTING_SO = ifelse(
      is.na(TEAM_BATTING_SO),
      median(TEAM_BATTING_SO, na.rm = TRUE),
      TEAM_BATTING_SO
    )
   )
```

When a distribution is observed to be skewed to the right, meaning small number of outliers has influence on the response variable `TARGET_WINS`, the log transformation can be applied to balance the scale of distribution. Utilizing log compresses the distribution of missing variable closer to symmetric shape. This makes the relationship with `TARGET_WINS` more linear and easier for the model to capture. `log1p` is used to handle any zero without producing undefined results.

```
# Log transform right-skewed variables
moneyball_clean <- moneyball_clean %>%
  mutate(
    LOG_TEAM_BASERUN_SB  = log1p(TEAM_BASERUN_SB),
    LOG_TEAM_PITCHING_SO = log1p(TEAM_PITCHING_SO),
    LOG_TEAM_FIELDING_E  = log1p(TEAM_FIELDING_E)
  )

moneyball_eval_clean <- moneyball_eval_clean %>%
  mutate(
    LOG_TEAM_BASERUN_SB  = log1p(TEAM_BASERUN_SB),
    LOG_TEAM_PITCHING_SO = log1p(TEAM_PITCHING_SO),
    LOG_TEAM_FIELDING_E  = log1p(TEAM_FIELDING_E)
  )
```

We can create three new variables that captures negative and positive impact on the wins for the team.

```
#  HR_DIFFERENTIAL and BAT_BB_SO_RATIO
moneyball_clean <- moneyball_clean |>
  mutate(
# Net home run differential (offensive power vs pitching vulnerability)
# The team that hits more home runs that it gives up will produce positive value
# the team that allows more home runs than it hits will produce a negative
#value, meaing they loosing the game
# Zero indicates the team is even in home runs
    HR_DIFFERENTIAL = TEAM_BATTING_HR - TEAM_PITCHING_HR,
  # Here we can measure how batter can judge whch pitch they would take a swing at
    # If the ratio is greater than 1, it reveals the team walk more than they strike
  # showing a more cohesive and patient team
    # This may happen if a batter force the pitchers to throw more pitches

    BAT_BB_SO_RATIO = ifelse(TEAM_BATTING_SO > 0,
                             TEAM_BATTING_BB / TEAM_BATTING_SO, NA)
  )

moneyball_eval_clean <- moneyball_eval_clean |>
  mutate(
    HR_DIFFERENTIAL = TEAM_BATTING_HR - TEAM_PITCHING_HR,
    BAT_BB_SO_RATIO = ifelse(TEAM_BATTING_SO > 0,
                             TEAM_BATTING_BB / TEAM_BATTING_SO, NA)
  )

#  Impute missing values in BAT_BB_SO_RATIO with median
moneyball_clean <- moneyball_clean |>
  mutate(
    BAT_BB_SO_RATIO = ifelse(is.na(BAT_BB_SO_RATIO),
                             median(BAT_BB_SO_RATIO, na.rm = TRUE), BAT_BB_SO_RATIO)
  )

moneyball_eval_clean <- moneyball_eval_clean |>
  mutate(
    BAT_BB_SO_RATIO = ifelse(is.na(BAT_BB_SO_RATIO),
                             median(BAT_BB_SO_RATIO, na.rm = TRUE), BAT_BB_SO_RATIO)
   )
```

Finally we can verify that there no missing variables that remain based on the results of our

Table 2

```
# Verify that no variables have remaining missingness
moneyball_clean |>
  summarise(across(everything(), ~ sum(is.na(.)))) |>
  tidyr::pivot_longer(everything(),
                      names_to  = "Variable",
                      values_to = "Missing") |>
  filter(Missing > 0) |>
  nrow()
```

```
[1] 0
```

imputation.

2.3 Verifying Linearity Assumption

Linear regression makes four important assumptions:

1. The independent and dependent variables are linearly related.
2. The errors are independent of each other.
3. The errors have constant variance.
4. The errors are normally distributed.

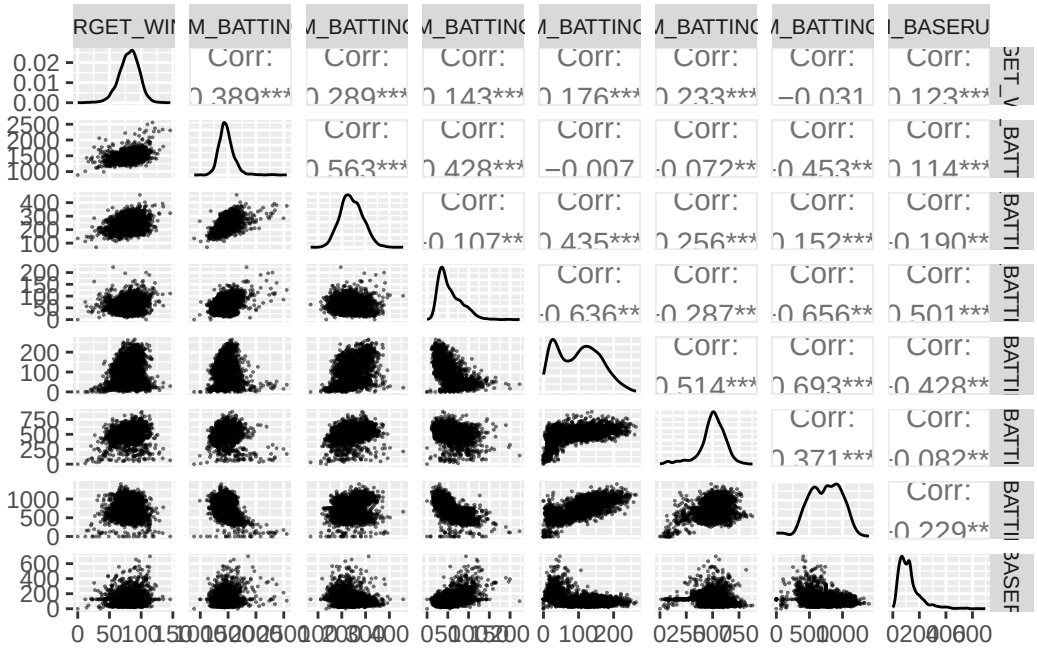
The last three will be evaluated after models are built (section 3). Here, we test the assumption of linearity using pair plots of the variables in the cleaned training dataset. To avoid overcrowded plots, we divided the variables into three groups.

2.3.1 Batting and baserun variables

- In the first row of Figure 1, the correlation coefficients show that all batting and baserun variables except `TEAM_BATTING_SO` have significant correlations with the target variable.
- In the first column, the scatterplots show that `TEAM_BATTING_H` and `TEAM_BATTING_2B` have the clearest linear relationship with `TARGET_WINS`.
- There are no obvious nonlinear relationships between batting and baserun variables and the target variable.

```
# This block takes a few seconds
moneyball_clean |>
  select(-c(ends_with('MISSING'))) |>  # omit categorical (flag) variables
  select(TARGET_WINS, starts_with('TEAM_BATTING'), TEAM_BASERUN_SB) |>
  ggpairs(
    # Color transparency and small data points help reduce overplotting
    mapping = aes(alpha = 0.1),
```

```
    lower = list(continuous = wrap('points', size = 0.01)),
    progress = FALSE
    ) +
    theme(
      strip.text = element_text(size = 8)
    )
```

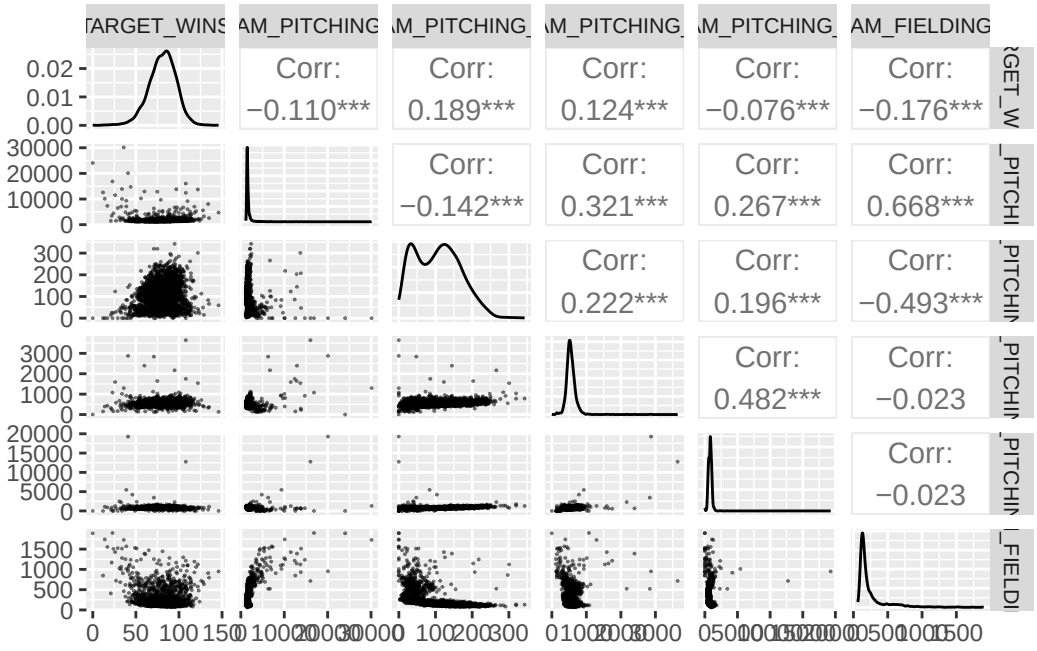


2.3.2 Pitching and fielding variables

- In the first row, the correlation coefficients show that all pitching variables and the fielding variable have significant correlations with the target variable.
- In the first column, the scatterplots show that `TEAM_PITCHING_BB` has the clearest linear relationship with `TARGET_WINS`.
  - Two independent variables (`TEAM_PITCHING_H` and `TEAM_PITCHING_SO`) appear to be related by a horizontal line. Technically, this satisfies the assumption of linearity, but it suggests that these variables do not have predictive value.
- There are no obvious nonlinear relationships between pitching and fielding variables and the target variable.

```
# This block takes a few seconds
moneyball_clean |>
  select(-c(ends_with('MISSING'))) |>  # omit categorical (flag) variables
  select(TARGET_WINS, starts_with('TEAM_PITCHING'), TEAM_FIELDING_E) |>
```

```
ggpairs(
  mapping = aes(alpha = 0.1),
  lower = list(continuous = wrap('points', size = 0.01)),
  progress = FALSE
  ) +
  theme(
    strip.text = element_text(size = 8)
  )
```

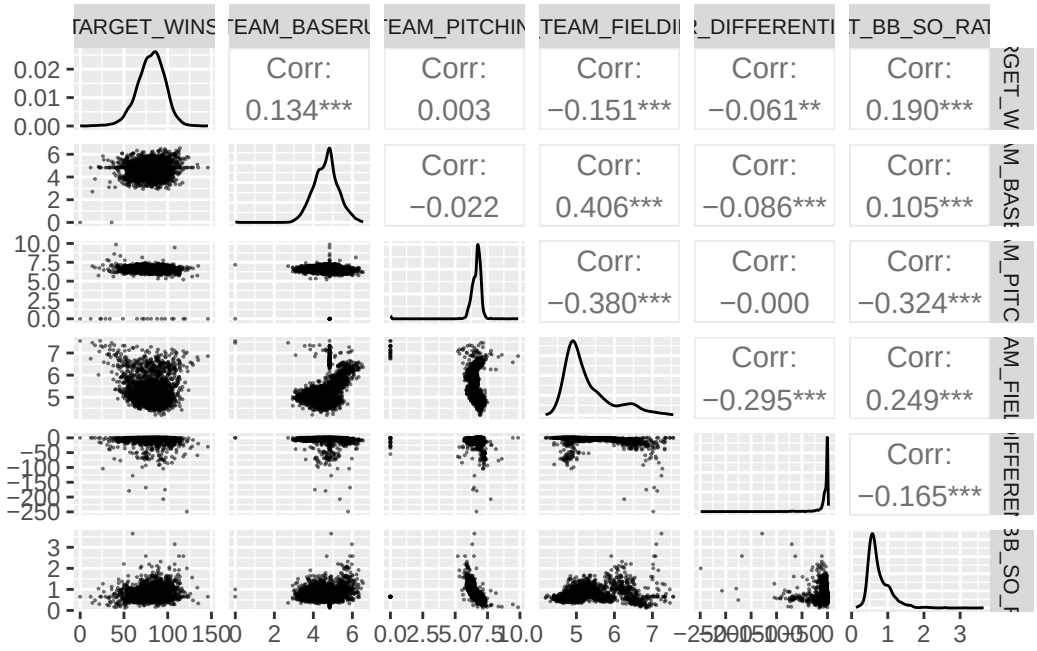


2.3.3 New variables

- In the first row, the correlation coefficients show that four of the five newly created variables have significant correlations with the target variable. The exception was `LOG_TEAM_PITCHING_SO`, which had a correlation coefficient of 0.003.
- In the first column, the scatterplots show that `BAT_BB_SO_RATIO` has the clearest linear relationship with `TARGET_WINS`.
  - `LOG_TEAM_PITCHING_SO` appears to be related by a horizontal line. Although this satisfies the assumption of linearity, it suggests that this variable may not have predictive value.
- There are no obvious nonlinear relationships between the newly created variables and the target variable.

```
# This block takes a few seconds
# Pair plots for new variables
```

```
moneyball_clean |>
  select(TARGET_WINS, starts_with('LOG'), HR_DIFFERENTIAL, BAT_BB_SO_RATIO) |>
  ggpairs(
    mapping = aes(alpha = 0.1),
    lower = list(continuous = wrap('points', size = 0.01)),
    progress = FALSE
    ) +
    theme(
      strip.text = element_text(size = 8)
    )
```



2.3.4 Linearity Assumption Summary

The results above indicate that the assumption of linearity is satisfied for all independent variables. However, several were flagged as being unlikely to have predictive value in a multivariate linear model.

2.4 Build Models

First, we will split our dataset into a trianing and testing set. We will use the training set to build our models and the testing set to evaluate their performance. The training set will be used to fit the models, while the testing set will be used to assess how well the models generalize to new data.

```
set.seed(19940211)
train_indices <- sample(nrow(moneyball_clean), size = 0.8 * nrow(moneyball_clean))
```

```
moneyball_train_split <- moneyball_clean[train_indices, ]
moneyball_test_split <- moneyball_clean[-train_indices, ]
```

We will start by building three models. The first model contains all the variables in the cleaned training dataset. The second model only includes the variables that are most correlated with the target variable and the two new predictors (`HR_DIFFERENTIAL` and `BAT_BB_SO_RATIO`). Finally, we'll create a third model using automated stepwise selection to identify the best subset of predictors. This method iteratively adds or removes predictors to find the model that best balances fit and complexity.

We will compare the performance of these two models to determine whether including all variables improves performance compared to a model with just the most correlated variables. The stepwise selection model will help us identify whether there are any additional predictors that improve performance beyond the most correlated variables and the new predictors. We will evaluate the performance of these models in the next section to determine which one is best for predicting `TARGET_WINS`.

```
# First model with all variables
model_full <- lm(
  TARGET_WINS ~ .,
  data = moneyball_train_split |>
    select(-INDEX)
  )

# Second model with most correlated variables and new predictors
model_theory <- lm(
  TARGET_WINS ~ TEAM_BATTING_H + TEAM_BATTING_2B + TEAM_BATTING_BB + HR_DIFFERENTIAL + BAT_BB_S
    data = moneyball_train_split
  )

# A third model using Automated Stepwise Selection
model_stepwise <- step(
  model_full,
  direction = "backward",
  trace = 0
)
```

2.4.1 Verifying Remaining Assumptions for Linear Regression

Prior to comparing the models, we will check the remaining assumptions of linear regression. Restating the assumptions:

2.4.1.1 The independent and dependent variables are linearly related.

This was verified above during the data exploration phase using pair plots. The scatterplots in the pair plots showed that there were no obvious nonlinear relationships between the independent variables and the target variable `TARGET_WINS`. Therefore, the assumption of linearity is satisfied.

2.4.1.2 The errors are independent of each other.

To check for independence of errors, we can plot the auto-correlations plots (ACF) of the residuals for each model. If there are significant autocorrelations at any lag, this suggests that the errors are not independent.

```
par(mfrow = c(1, 3))
acf(residuals(model_full), main="ACF: Full Model")
acf(residuals(model_theory), main="ACF: Theory Model")
acf(residuals(model_stepwise), main="ACF: Stepwise Model")
```

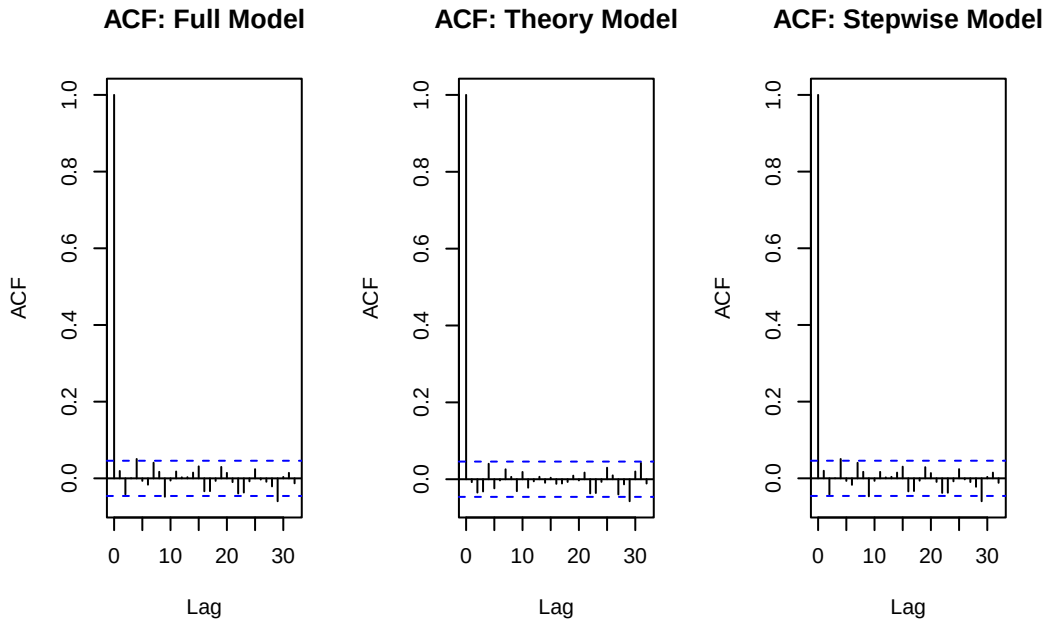


Figure 4: ACF plots of residuals for each model

These ACF plots show that there are just a few significant autocorrelations from lag 1 to lag 10 for all three models, which suggests that the errors are not independent. This may be due to the fact that the data is time series data, and there may be temporal dependencies in the errors. This would make sense as the performance of a baseball team in one year may be related to its performance in the previous year and this data is defined as "...[representing] a professional baseball team from the years 1871 to 2006 inclusive. Each record has the performance of the team for the given year, with all of the statistics adjusted to match the performance of a 162 game season."

2.4.1.3 The errors have constant variance (Homoscedasticity).

To check for constant variance, we examine the Residuals vs Fitted plots and the Scale-Location plots for our models. We are looking for a relatively random scatter of points around the horizontal zero line. If the spread of the residuals increases or decreases systematically with the fitted values (forming a "megaphone" or "funnel" shape), it indicates heteroscedasticity.

```
# Set up a 2x3 grid for the plots
par(mfrow = c(2, 3))

# Row 1: Residuals vs Fitted
plot(model_full, which = 1, main = "Resid vs Fitted (Full)")
plot(model_theory, which = 1, main = "Resid vs Fitted (Theory)")
plot(model_stepwise, which = 1, main = "Resid vs Fitted (Step)")

# Row 2: Scale-Location
plot(model_full, which = 3, main = "Scale-Location (Full)")
plot(model_theory, which = 3, main = "Scale-Location (Theory)")
plot(model_stepwise, which = 3, main = "Scale-Location (Step)")
```

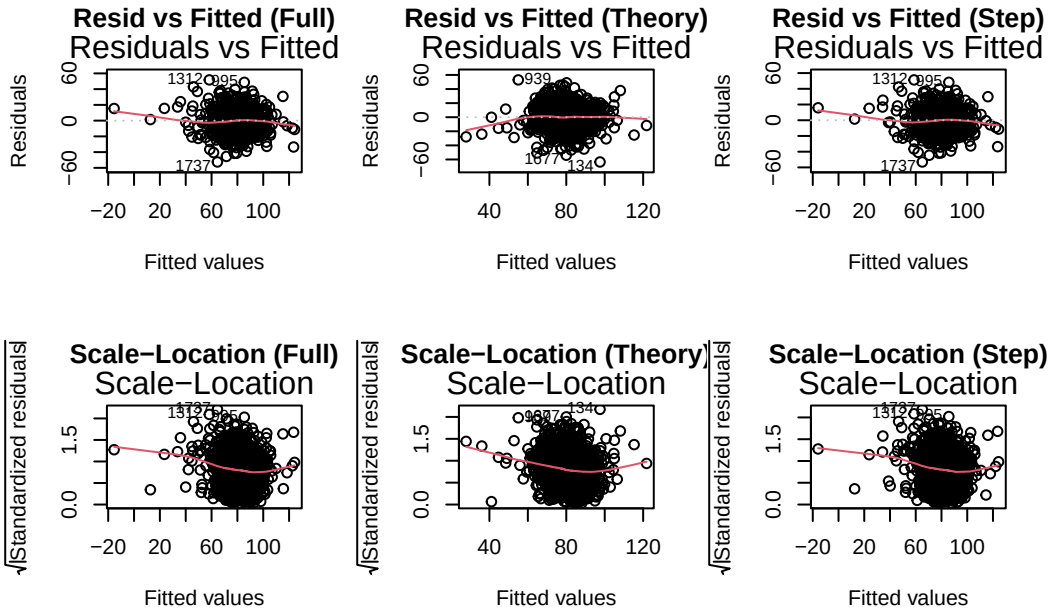


Figure 5: Residuals vs Fitted and Scale-Location plots for each model

The residuals vs fitted plots across all three models show that the residuals are randomly scattered around the horizontal zero line, which is evidence that the model adheres to the assumption of homoscedasticity. The scale-location plots show a slightly different story where theres a consistant

'U' or 'V' shape with the red line indicating that the variance of the residuals is not constant across the range of fitted values. This suggests that there may be some heteroscedasticity in the models, which could affect the reliability of the standard errors and confidence intervals.

2.4.1.4 The errors are normally distributed.

To verify that our residuals follow a normal distribution, we use Normal Q-Q plots. If the assumption holds then the standardized residuals should be in a diagonal line. Deviations at the lower or upper tails suggest skewed or heavy-tailed errors. We will also perform a Shapiro-Wilk statistical test to quantitatively assess normality, where a p-value < 0.05 indicates a significant departure from a normal distribution.

```
# Set up a 1x3 grid for the plots
par(mfrow = c(1, 3))
plot(model_full, which = 2, main = "Normal Q-Q (Full)")
plot(model_theory, which = 2, main = "Normal Q-Q (Theory)")
plot(model_stepwise, which = 2, main = "Normal Q-Q (Step)")
```

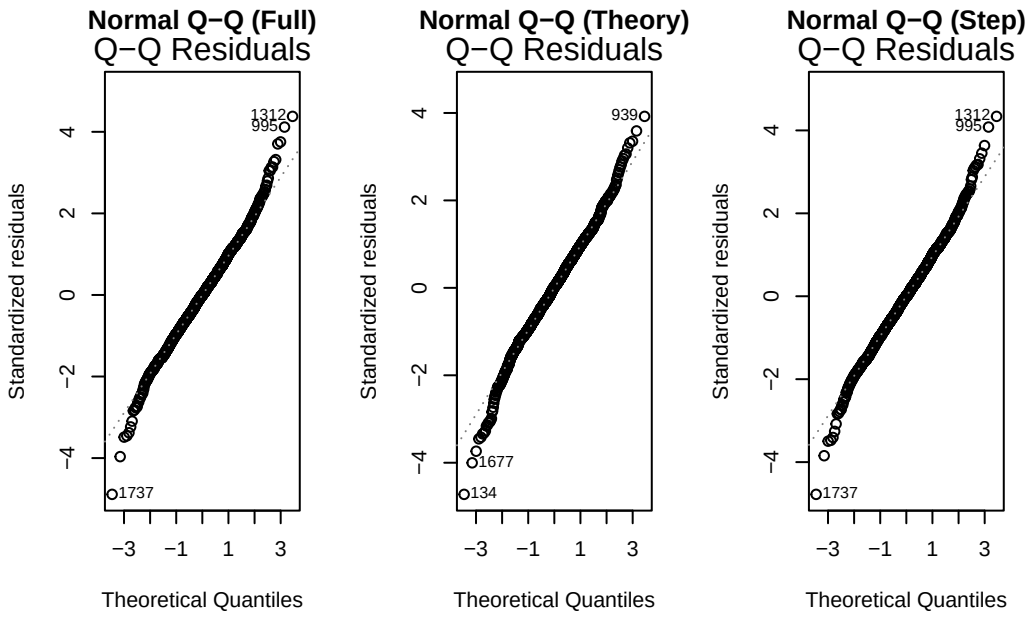


Figure 6: Normal Q-Q plots for checking normality of errors

```
# Formal Shapiro-Wilk Test (Note: p-value < 0.05 means NOT normal)
shapiro.test(residuals(model_full))
```

```
    Shapiro-Wilk normality test
```

```
data:  residuals(model_full)
W = 0.99597, p-value = 9.012e-05
```

```
shapiro.test(residuals(model_theory))
```

```
    Shapiro-Wilk normality test

data:  residuals(model_theory)
W = 0.99313, p-value = 1.701e-07
```

```
shapiro.test(residuals(model_stepwise))
```

```
    Shapiro-Wilk normality test

data:  residuals(model_stepwise)
W = 0.99609, p-value = 0.0001211
```

For all 3 models the points follow the diagonal line reasonably well in the middle of the distribution but the upper tail curves upwards and the lower tail curves downwards. This suggests that the residuals have heavier tails than a normal distribution, which may indicate the presence of outliers or a non-normal error distribution.

From the Shapiro-Wilk tests, we find that the p-values for all three models are much less than 0.05 which indicates that the residuals significantly deviate from a normal distribution. This is consistent with the observations from the Q-Q plots and suggests that the assumption of normality is violated for all three models. This violation is likely due to the presence of outliers or a heavy-tailed error distribution and we can likly continue with the models but we should be cautious when interpreting the results.

2.5 Select Models

```
# Creating a custom helper function to extract all the required metrics
# explicitly evaluating on the provided training dataset.
get_model_metrics <- function(model, model_name, train_data) {
  mod_summary <- summary(model)

  # Explicitly calculate Mean Squared Error (MSE) using the training data
  predictions <- predict(model, newdata = train_data)
  mse <- mean((train_data$TARGET_WINS - predictions)^2)
```

```r
  # Extract F-statistic and calculate its p-value (inherent to training data)
  f_stat <- mod_summary$fstatistic
  p_val <- if (!is.null(f_stat)) {
    pf(f_stat[1], f_stat[2], f_stat[3], lower.tail = FALSE)
  } else {
    NA
  }

  # Return a nicely formatted row for our table
  tibble(
    Model = model_name,
    `Training MSE` = mse,
    `R-Squared` = mod_summary$r.squared,
    `Adj R-Squared` = mod_summary$adj.r.squared,
    `F-Statistic` = f_stat[1],
    `P-Value` = p_val,
    `AIC` = AIC(model),
    `BIC` = BIC(model)
  )
}

# Apply the function to all three models, explicitly passing the training split
model_comparison <- bind_rows(
  get_model_metrics(model_full, "Full Model", moneyball_train_split),
  get_model_metrics(model_theory, "Theory Model", moneyball_train_split),
  get_model_metrics(model_stepwise, "Stepwise Model", moneyball_train_split)
) |>
  # Sort by the lowest Training MSE
  arrange(`Training MSE`)

# Display the table
knitr::kable(
  model_comparison,
  digits = 4,
  caption = "Evaluation of Candidate Models on Training Dataset"
)
```

Table 3: Evaluation of Candidate Models on Training Dataset

Model	Training MSE	R-Squared	Adj R-Squared	F-Statistic	P-Value	AIC	BIC
Full Model	144.3165	0.4058	0.3999	68.3422	0	14253.99	14364.12
Stepwise Model	144.3856	0.4056	0.4009	87.9598	0	14246.86	14334.97
Theory Model	185.8744	0.2347	0.2326	111.2877	0	14688.57	14727.11

We favor a slightly more parsimonious model over a complex one with a marginally better R2. A simpler model (like our Stepwise or Theory model) is easier to interpret and less prone to overfitting. We will evaluate the models using Adjusted `R^2` and MSE on the training data, and then use the evaluation dataset to compare their predictive performance.

Looking at the table above, we can see that the Stepwise and Full models have similar MSE and Adjusted `R^2`, but the Stepwise model has fewer predictors making it a bit more parsimonious. Additionally, the Stepwise model is more interpretable and less prone to overfitting than the Full model. Additonally, the theory model has a much higher MSE and lower Adjusted `R^2` than the other two models, which suggests that baseball is more comoplex to predict wins with just the most correlated variables and the two new predictors. Therefore, **we select the Stepwise model as our final model for predicting `TARGET_WINS`**.

Regarding the F-statistic, the highly significant p-value for the stepwise model's F-statistic (119.9) confirms that our selected group of predictors is collectively highly effective at explaining the variance in total team wins.

```
# | label: stepwise-model-summary
# | echo: true
# Display the summary of the selected Stepwise model to interpret coefficients and significanc
summary(model_stepwise)
```

```
Call:
lm(formula = TARGET_WINS ~ TEAM_BATTING_H + TEAM_BATTING_2B +
    TEAM_BATTING_3B + TEAM_BATTING_HR + TEAM_BATTING_BB + TEAM_BATTING_SO +
    TEAM_BASERUN_SB + TEAM_PITCHING_H + TEAM_PITCHING_BB + TEAM_FIELDING_E +
    TEAM_BASERUN_SB_MISSING + TEAM_BATTING_SO_MISSING + LOG_TEAM_FIELDING_E +
    BAT_BB_SO_RATIO, data = select(moneyball_train_split, -INDEX))

Residuals:
```

```
    Min      1Q  Median      3Q     Max
-53.105  -7.792   0.033   7.796  51.332

Coefficients:
                          Estimate Std. Error t value Pr(>|t|)
(Intercept)              7.343e+01  1.192e+01   6.161 8.88e-10 ***
TEAM_BATTING_H           4.931e-02  3.894e-03  12.662  < 2e-16 ***
TEAM_BATTING_2B         -4.184e-02  9.628e-03  -4.346 1.47e-05 ***
TEAM_BATTING_3B          8.995e-02  1.804e-02   4.986 6.74e-07 ***
TEAM_BATTING_HR          4.927e-02  1.050e-02   4.692 2.92e-06 ***
TEAM_BATTING_BB          2.900e-02  5.654e-03   5.129 3.22e-07 ***
TEAM_BATTING_SO         -2.113e-02  3.329e-03  -6.347 2.78e-10 ***
TEAM_BASERUN_SB          6.328e-02  4.726e-03  13.390  < 2e-16 ***
TEAM_PITCHING_H          7.941e-04  4.149e-04   1.914  0.05579 .
TEAM_PITCHING_BB         5.104e-03  2.888e-03   1.767  0.07735 .
TEAM_FIELDING_E         -2.436e-02  5.581e-03  -4.365 1.34e-05 ***
TEAM_BASERUN_SB_MISSING  3.191e+01  2.079e+00  15.349  < 2e-16 ***
TEAM_BATTING_SO_MISSING  1.244e+01  1.556e+00   7.995 2.29e-15 ***
LOG_TEAM_FIELDING_E     -1.273e+01  2.133e+00  -5.966 2.92e-09 ***
BAT_BB_SO_RATIO         -7.029e+00  2.473e+00  -2.843  0.00452 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 12.07 on 1805 degrees of freedom
Multiple R-squared:  0.4056,    Adjusted R-squared:  0.4009
F-statistic: 87.96 on 14 and 1805 DF,  p-value: < 2.2e-16
```

To make inferences from our Stepwise model, we can interpret the coefficients. For example, the coefficient for `TEAM_BATTING_H` is 0.05 which indicates that, all other things being equal, for every additional 20 hits a team records over a season, they're expected to gain about 1 additional win. Similarly, `TEAM_BATTING_BB` (i.e., walks) has a positive highly significant coefficient (0.03) which aligns with baseball theory that teams that walk more tend to win more.

3 Results

```r
# Generate predictions on the evaluation dataset using the selected model
eval_predictions <- predict(model_stepwise, newdata = moneyball_eval_clean)

# Bind the predictions back to the evaluation dataset for easy viewing
```

```
moneyball_eval_results <- moneyball_eval_clean |>
  # We round the predictions to the nearest whole number because a team
  # can't win a fraction of a baseball game
  mutate(PREDICTED_TARGET_WINS = round(eval_predictions, 0)) |>
  # Move the new prediction column to the front (right after INDEX) for easy reading
  relocate(PREDICTED_TARGET_WINS, .after = INDEX)

# Write predictions to a csv
write_csv(moneyball_eval_results, "moneyball_evaluation_predictions.csv")

# Display the first 10 predictions in the rendered PDF
knitr::kable(
  head(moneyball_eval_results |> select(INDEX, PREDICTED_TARGET_WINS), 10),
  caption = "Excerpts of Stepwise Model Win Predictions for Evaluation Set"
)
```

Table 4: Excerpts of Stepwise Model Win Predictions for Evaluation Set

INDEX	PREDICTED_TARGET_WINS
9	61
10	67
14	74
47	86
60	82
63	83
74	76
83	67
98	69
120	71

Given our evaluation set and our stepwise model, we are able to create predictions for the number of wins for the team in each year. The first 10 predictions are shown in the table above. We write these predictions to a CSV file for submission.

4 Conclusion

In conclusion, our stepwise model performed best in predicting the number of baseball wins based on the training data. Since there were some minor violations of assumptions, these results should be interpreted with caution, which may affect the reliability of our model. Future work could explore

data transformations or alternative modeling approaches to address these assumption violations and potentially improve predictive performance.